Chapter 1

Building `fall_n` on Windows (MSYS2/UCRT64)

This chapter documents the full procedure to configure, compile, and run `fall_n` on a 64-bit Windows machine using the MSYS2 UCRT64 toolchain. Every step is reproducible from a clean Windows install and does not require Visual Studio, WSL, or any other Unix compatibility layer beyond MSYS2 itself.

1.1 Prerequisites and Motivations

`fall_n` targets C++23 and depends on several scientific libraries (PETSc, Eigen, VTK, MS-MPI) that are either unavailable or extremely inconvenient to build from source under the MSVC ABI. The chosen toolchain is:

- **MSYS2 UCRT64** — a MinGW-w64 environment based on the Universal C Runtime (UCRT), which ships as a standard component of modern Windows. All packages are installed via the `pacman` package manager, giving reproducible binary packages without local compilation.
- **GCC 15.2** — full C++23 support including `<print>`, `std::expected`, ranges, and concepts.
- **Ninja 1.13 + CMake 4.2** — fast incremental builds.
- **Microsoft MPI (MS-MPI)** — the official MPI runtime for Windows, required by PETSc at link time and at runtime.

The companion library versions used and verified to work are summarised in **??**.

1.2 Step 1 — Install MSYS2

Download the MSYS2 installer from `https://www.msys2.org` and run it. The default installation path is `C:\msys64`. After installation, open the **MSYS2 UCRT64** terminal (not the plain MSYS2 terminal) from the Start menu.

Update the base system before installing anything:

```
pacman -Syu          # update database + core packages; close and reopen
pacman -Su           # finish updating remaining packages
```

Table 1.1: Dependency versions verified on Windows (MSYS2 UCRT64, March 2026).

Library	MSYS2 package name	Version	Notes
GCC	`mingw-w64-ucrt-x86_64-gcc`	15.2.0	C++23
CMake	`mingw-w64-ucrt-x86_64-cmake`	4.2.3	
Ninja	`mingw-w64-ucrt-x86_64-ninja`	1.13.2	
pkg-config	`mingw-w64-ucrt-x86_64-pkgconf`	2.x	
Eigen	`mingw-w64-ucrt-x86_64-eigen3`	5.0.1	header-only
PETSc	`mingw-w64-ucrt-x86_64-petsc`	3.24.5	double/MPI/opt
VTK	`mingw-w64-ucrt-x86_64-vtk`	9.5.2	
MS-MPI SDK	`mingw-w64-ucrt-x86_64-msmpi`	10.1.1	headers+stubs
nlohmann-json	`mingw-w64-ucrt-x86_64-nlohmann-json`	3.12.0	VTK dep
fast_float	`mingw-w64-ucrt-x86_64-fast_float`	8.2.4	VTK dep
utf8cpp	`mingw-w64-ucrt-x86_64-utf8cpp`	4.0.6	VTK dep
MS-MPI runtime	`winget: Microsoft.msmpi`	10.1.12	runtime DLL

1.3 Step 2 — Install Build Tools and Dependencies

All subsequent commands are executed inside the **MSYS2 UCRT64** terminal. Install the compiler toolchain and build tools first:

```
pacman -S \
  mingw-w64-ucrt-x86_64-gcc \
  mingw-w64-ucrt-x86_64-cmake \
  mingw-w64-ucrt-x86_64-ninja \
  mingw-w64-ucrt-x86_64-pkgconf
```

Then install the scientific libraries and their transitive dependencies:

```
pacman -S \
  mingw-w64-ucrt-x86_64-eigen3 \
  mingw-w64-ucrt-x86_64-petsc \
  mingw-w64-ucrt-x86_64-vtk \
  mingw-w64-ucrt-x86_64-msmpi \
  mingw-w64-ucrt-x86_64-nlohmann-json \
  mingw-w64-ucrt-x86_64-fast_float \
  mingw-w64-ucrt-x86_64-utf8cpp
```

Remark 1.1. The packages `nlohmann-json`, `fast_float`, and `utf8cpp` are direct dependencies of the MSYS2 VTK 9.5.2 package and must be installed separately — they are not pulled in automatically. Omitting them causes CMake's `find_package(VTK)` to fail with missing component errors.

Remark 1.2. The MSYS2 PETSc package uses the pkg-config name `petsc-dmo` (double-precision, MPI-enabled, optimized build). The legacy name `PETSc` used in the original Linux `CMakeLists.txt` does not exist; see **??** for the necessary `pkg_search_module` adjustment.

1.4 Step 3 — Install the MS-MPI Runtime

The `mingw-w64-ucrt-x86_64-msmpi` pacman package provides the MS-MPI *SDK* (headers and import libraries) needed to compile and link MPI programs. However, the *runtime DLL* (`msmpi.dll`) is distributed separately by Microsoft and must be installed on every machine that will *run* executables linked against MS-MPI (including during `ctest`).

Install the runtime using winget from a regular Windows PowerShell or Command Prompt (not from inside MSYS2):

```
winget install Microsoft.msmpi --accept-package-agreements
```

This installs `mpiexec.exe` to `C:\Program Files\Microsoft MPI\Bin\` and registers `msmpi.dll` in the Windows system search path. Without this step all PETSc-linked executables will abort at startup with exit code `0xc0000135` (DLL NOT FOUND).

1.5 Step 4 — Configure the CMake Build

Clone (or copy) the repository to a convenient location, e.g. `C:\MyLibs\fall_n`. All remaining commands are executed inside the **MSYS2 UCRT64** terminal.

Create a build directory, configure, and generate:

```
cd /c/MyLibs/fall_n
mkdir -p build && cd build
cmake .. --preset win-debug
```

The `win-debug` preset is defined in `CMakePresets.json` and activates only on Windows hosts. It selects the UCRT64 Ninja generator and sets `CMAKE_BUILD_TYPE=Debug`.

A successful configuration ends with output similar to:

```
-- Configuring done (3.6s)
-- Generating done (0.5s)
-- Build files have been written to: C:/MyLibs/fall_n/build
```

1.6 Step 5 — Compile

From the build directory, invoke Ninja:

```
ninja -j8          # build everything (main + seismic + all 48 tests)
ninja -j8 fall_n   # build only the main executable
```

On a modern 8-core machine the full build (including the Matplot++ dependency fetched via `FetchContent`) takes approximately 4–6 minutes. The following executables are produced in the build directory:

- `fall_n.exe` — main structural analysis executable;
- `fall_n_seismic.exe` — seismic analysis entry point;
- 48 test executables (`fall_n_*_test.exe`).

1.7 Step 6 — Run the Tests

```
ctest -L unit -j4              # fast unit tests only (~4 s)
ctest -L 'unit|integration' -j4
ctest -j4                      # all 48 tests
```

Expected results on a clean install: **44/48 tests pass**. The 4 remaining failures are not related to the Windows port; they are caused by missing mesh and seismic data files that are not included in the repository:

- `seismic_infrastructure`: requires `data/input/earthquakes/el_centro_1940_ns.dat`;

- `nonlinear_validation`, `vertical_slice_gate`: require `tests/validation_cube.msh`;
- `boundary_conditions`: requires `data/input/Beam_LagCell_Ord1_30x6x3.msh`.

1.8 CMake Modifications for Windows Compatibility

This section documents every change that was made to `CMakeLists.txt` and `CMakePresets.json` relative to the original Linux build system. All changes are guarded by platform conditions so the file remains fully functional on Linux without modification.

1.8.1 MinGW-specific compiler and linker flags

GCC on MinGW-w64 requires two flags that are not needed on Linux:

1. **`-lstdc++exp`** — The C++23 `<print>` header (`std::print`, `std::println`) is implemented in a separate experimental static library on MinGW. Without this flag all translation units that include `<print>` fail at link time with undefined references to `_ZSt5printB5cxx11...`.
2. **`-Wa,-mbig-obj`** — The PE/COFF object file format (used on Windows) limits the number of sections in a single `.obj` file to 65 535. Template-heavy translation units in `fall_n` (particularly those including PETSc and VTK headers) exceed this limit and cause the assembler to abort with the error *"too many sections (73659 > 65535)"*. This flag enables the *big-obj* format which raises the limit to over four billion sections.

The corresponding CMake block added to `CMakeLists.txt` is:

```
if(MINGW AND CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
  link_libraries(stdc++exp)
  add_compile_options(-Wa,-mbig-obj)

  # VTK DLLs on MinGW do not export inline template member functions.
  # Predefine these guards to skip the extern template declarations
  # and allow the compiler to instantiate the functions locally.
  add_compile_definitions(
    VTK_AOS_DATA_ARRAY_TEMPLATE_EXTERN
    VTK_GDA_TEMPLATE_EXTERN
  )
endif()
```

1.8.2 VTK extern-template linking fix

This is the most subtle Windows-specific issue. VTK uses *explicit template instantiation* to reduce compile times: the class template `vtkAOSDataArrayTemplate<T>` (and its ancestor `vtkGenericDataArray`
is fully instantiated inside the `libvtkCommonCore.dll` shared library, while all consumer translation units use `extern template` declarations (gated by the macro `VTK_USE_EXTERN_TEMPLATE`, unconditionally defined in `vtkCompiler.h`) to suppress local re-instantiation.

On Linux and macOS this mechanism works perfectly: the shared library exports the symbols and the dynamic linker resolves them at load time. On Windows with MinGW, however, the `extern template` suppression interacts poorly with the PE/COFF `__declspec(dllimport)` mechanism. In particular, *inline methods* such as

```
// vtkAOSDataArrayTemplate.h (line 87)
void SetValue(vtkIdType valueIdx, ValueType value) {
  this->Buffer->GetBuffer()[valueIdx] = value;
}
```

are *already inlined* into the DLL's callers at the time the DLL is built; they are *not emitted as distinct exported symbols*. Yet `extern template class VTKCOMMONCORE_EXPORT vtkAOSDataArrayTemplate<` instructs the consumer translation unit to *not* instantiate the template locally and to expect the symbol from the DLL import library — a symbol that does not exist. The linker then fails with:

```
undefined reference to
  `vtkAOSDataArrayTemplate<long long>::SetValue(long long, long long)'
undefined reference to
  `vtkAOSDataArrayTemplate<int>::SetValue(int, int)'
undefined reference to
  `vtkAOSDataArrayTemplate<double>::SetValue(double, double)'
```

The fix is to predefine the guard macros `VTK_AOS_DATA_ARRAY_TEMPLATE_EXTERN` and `VTK_GDA_TEMPLATE_E` before any VTK header is processed. These macros wrap the `extern template` declarations inside `#ifndef` guards:

```
// vtkAOSDataArrayTemplate.h (abbreviated)
#ifdef VTK_USE_EXTERN_TEMPLATE
#ifndef VTK_AOS_DATA_ARRAY_TEMPLATE_EXTERN   // <-- guard
#define VTK_AOS_DATA_ARRAY_TEMPLATE_EXTERN
extern template class VTKCOMMONCORE_EXPORT vtkAOSDataArrayTemplate<
   double>;
// ... 12 more explicit instantiations ...
#endif
#endif
```

By predeclaring `VTK_AOS_DATA_ARRAY_TEMPLATE_EXTERN` (and the analogous `VTK_GDA_TEMPLATE_EXTERN` for the base class), the `extern template` block is skipped and the compiler instantiates the inline methods locally from the `.h` + `.txx` pair shipped with the package. This is the correct semantics on MinGW: small inline methods should never have been placed behind DLL-import stubs in the first place.

1.8.3 PETSc C++ macro fallbacks

The MSYS2 PETSc 3.24.5 package was compiled as a pure-C library (`PETSC_CLANGUAGE_C = 1`). As a result, two macros required by the C++ PETSc headers are absent from `petscconf.h`:

- `PETSC_FUNCTION_NAME_CXX` — the C++ equivalent of `__FUNCTION__`, used in error-handling macros such as `PetscCall` for stack-trace annotation.
- `PETSC_CXX_RESTRICT` — the C++ spelling of the `restrict` qualifier.

Missing either macro causes compile errors of the form *"PETSC_FUNCTION_NAME_CXX was not declared"*. The fix uses CMake's `CheckSymbolExists` to detect their absence and defines fallbacks:

```
include(CheckSymbolExists)
set(CMAKE_REQUIRED_INCLUDES ${PETSC_INCLUDE_DIRS})
check_symbol_exists(PETSC_FUNCTION_NAME_CXX "petscconf.h"
                    _HAVE_PETSC_FN_CXX)
if(NOT _HAVE_PETSC_FN_CXX)
  add_compile_definitions(
    PETSC_FUNCTION_NAME_CXX=__PRETTY_FUNCTION__)
endif()
check_symbol_exists(PETSC_CXX_RESTRICT "petscconf.h"
                    _HAVE_PETSC_CXX_RESTRICT)
if(NOT _HAVE_PETSC_CXX_RESTRICT)
```

```
  add_compile_definitions(PETSC_CXX_RESTRICT=__restrict__)
endif()
```

1.8.4 PETSc pkg-config name

The original `CMakeLists.txt` searched for PETSc via `pkg_search_module(PETSC REQUIRED IMPORTED_TARGET PETSc)`. The MSYS2 package registers the library under the name `petsc-dmo` (double, MPI, optimized). The search was widened to try multiple names:

```
pkg_search_module(PETSC REQUIRED IMPORTED_TARGET
                  PETSc petsc petsc-dmo petsc-dso)
```

1.8.5 Source directory macro

Hardcoded absolute Unix paths of the form `/home/sechavarriam/MyLibs/fall_n/` existed in `main.cpp`, `main_seismic.cpp`, and thirteen test files. These were replaced with a compile-time macro that resolves to the actual source tree regardless of platform or user name:

```
# CMakeLists.txt
add_compile_definitions(
  FALL_N_SOURCE_DIR="${CMAKE_SOURCE_DIR}")
```

Usage in source code:

```
#ifdef FALL_N_SOURCE_DIR
  static const std::string BASE =
      std::string(FALL_N_SOURCE_DIR) + "/";
#else
  static const std::string BASE = "./";
#endif
```

1.8.6 Windows CMake preset

A `win-debug` preset was added to `CMakePresets.json`:

```
{
  "name": "win-debug",
  "displayName": "Windows Debug (MSYS2 UCRT64)",
  "generator": "Ninja",
  "binaryDir": "${sourceDir}/build",
  "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug" },
  "condition": {
    "type": "equals",
    "lhs": "${hostSystemName}",
    "rhs": "Windows"
  }
}
```

1.9 Source-Code Portability Fixes

Besides the build-system changes, several source files required modifications to compile cleanly with GCC on Windows.

1.9.1 `ushort` is not a standard type

The file `src/utils/index.hh` used `ushort` as a template non-type parameter:

```
template <ushort... N>  // non-standard: ushort is a POSIX extension
```

GCC on MinGW does not declare `ushort` in the global namespace. The fix replaces it with the standard `unsigned short`:

```
template <unsigned short... N>
```

1.9.2 `%zu` format specifier in `PetscPrintf`

The MSYS2 PETSc runtime's `PetscPrintf` does not recognise the `%zu` format specifier for `std::size_t` on this platform. Occurrences were replaced with `%d` and an explicit `(int)` cast:

```
// Before
PetscPrintf(PETSC_COMM_WORLD,
    "T_measured = %.6f (from %zu crossings)\n",
    T_measured, crossings.size());

// After
PetscPrintf(PETSC_COMM_WORLD,
    "T_measured = %.6f (from %d crossings)\n",
    T_measured, (int)crossings.size());
```

1.9.3 Unused-variable warning in tests

GCC 15 with `-Werror=unused-but-set-variable` rejected a variable in `tests/test_kinematic_policy.cpp` that was assigned but never read after the assignment. The fix adds `[[maybe_unused]]`:

```
[[maybe_unused]] Eigen::Vector<double, 3> E_from_B = gp.B * u_e;
```

1.10 Runtime PATH Considerations

The UCRT64 runtime libraries (`libvtkCommonCore.dll`, `libpetsc-dmo.dll`, `libgcc_s_seh-1.dll`, etc.) reside in `C:\msys64\ucrt64\bin`. This directory is automatically on `PATH` inside MSYS2 terminal sessions.

When executables are launched *outside* MSYS2 (e.g. from PowerShell, Explorer, or a CI runner) they will fail with `0xc0000135` unless the UCRT64 bin path is added to the system or user `PATH`, or the DLLs are copied alongside the executables.

For CTest invocations the recommended approach is to run from within the MSYS2 UCRT64 terminal, where the path is already configured:

```
# Open MSYS2 UCRT64 terminal, then:
cd /c/MyLibs/fall_n/build
ctest -L unit -j4 --output-on-failure
```

Alternatively, add the UCRT64 bin directory permanently to the Windows system environment variables via *System Properties* → *Environment Variables* → *System Variables* → *Path*.

1.11 Summary of All Changes

?? lists every file that was modified and the corresponding reason.

Table 1.2: Files modified for Windows/MinGW compatibility.

File	Change
`CMakeLists.txt`	MinGW flags (`-lstdc++exp`, `-Wa,-mbig-obj`); PETSc fallback macros; `pkg_search_module` widened; VTK extern-template guards; `FALL_N_SOURCE_DIR` macro; compiler flag guard for MSVC vs. GCC.
`CMakePresets.json`	Added `win-debug` preset.
`main.cpp`	Replaced hardcoded `/home/...` path with `FALL_N_SOURCE_DIR` macro.
`main_seismic.cpp`	Same as `main.cpp`.
`src/utils/index.hh`	`ushort` → `unsigned short`.
13 test files under `tests/`	Replaced hardcoded Unix paths with `FALL_N_SOURCE_DIR`; replaced `/tmp/` with `std::filesystem::temp_directory_path()`.
`tests/test_kinematic_policy.cpp`	Added `[[maybe_unused]]` to suppress GCC 15 unused-variable warning.
`tests/test_phase5_dynamic_verification.cpp`	Replaced `%zu` with `%d` in `PetscPrintf` format string.